MINI COMPILER FOR ARITHMETIC EXPRESSION

REVIEW – 1

Team members

Roll No	Name
2420030592	J.Kumudavalli
2420030415	G.Rithwik

Under the guidance of

Dr. K.Krishna kumar

Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Table of contents

- Abstract
- Introduction
- Objectives
- Literature Survey
- Implementation
- Advantages and limitations
- Conclusion

Abstract

The Mini Compiler for Arithmetic Expressions is a simplified compiler designed to demonstrate the fundamental concepts and phases of compiler design. The system accepts arithmetic expressions containing operands, arithmetic operators such as addition, subtraction, multiplication, and division, along with parentheses. The input expression is first processed through lexical analysis, where it is divided into meaningful tokens such as numbers, operators, and parentheses. These tokens are then passed to the syntax analysis phase, which checks whether the expression follows the defined grammar and verifies the correct arrangement of operands and operators. Semantic analysis performs basic validity checks and identifies invalid operations. After successful analysis, the compiler generates an intermediate representation such as Three-Address Code (TAC). The system also provides error detection and reporting for invalid expressions. The project demonstrates how a high-level arithmetic expression can be transformed step by step into an intermediate form and provides practical understanding of tokens, grammar, parsing, syntax trees, semantic analysis, intermediate code generation, and error handling.

Introduction

- A compiler is a system software that translates a high-level program into a form that can be understood and executed by a computer. The compilation process involves several important phases, such as lexical analysis, syntax analysis, semantic analysis, and intermediate code generation.
- The Mini Compiler for Arithmetic Expressions is designed to demonstrate these fundamental compiler phases using simple arithmetic expressions. It accepts expressions containing operands, arithmetic operators such as addition, subtraction, multiplication, and division, along with parentheses. The input is first divided into meaningful tokens during lexical analysis. These tokens are then checked by the syntax analyzer to verify the correct arrangement of operators and operands. Basic semantic checks are performed, and finally, the valid expression is converted into an intermediate representation such as Three-Address Code. The project also includes error detection and reporting for invalid expressions, providing a practical understanding of how arithmetic expressions are processed inside a compiler.

Objectives

The main objectives of the Mini Compiler for Arithmetic Expressions are:
To understand the basic phases of compiler design.
To develop a system that accepts arithmetic expressions as input.
To perform lexical analysis by identifying numbers, identifiers, operators, and parentheses.
To perform syntax analysis and verify whether an expression follows the specified grammar.
To check the correct arrangement of operands and operators.
To perform basic semantic analysis and identify invalid operations.
To handle arithmetic operators such as addition, subtraction, multiplication, and division.
To support parentheses and proper operator precedence.
To generate an intermediate representation such as Three-Address Code.
To detect and report syntax and semantic errors.
To understand how high-level arithmetic expressions are converted into simpler intermediate instructions.
To provide an educational model for understanding the internal working of a compiler.

Literature Survey

S. No.	Authors	Year	Title / Work	Methodology / Contribution	Relevance to Our Project
1	Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman	2007	*Compilers: Principles, Techniques, and Tools*	Presents the major phases of compiler design including lexical analysis, syntax analysis, semantic analysis, intermediate code generation, and optimization.	Provides the theoretical foundation for implementing the phases of our mini compiler.
2	D. Gries	1978	*Theoretical Foundations for Compiler Construction*	Discusses compiler construction techniques and analyzes parsing approaches using grammars and arithmetic expressions.	Helps in understanding grammar-based parsing and expression processing.
3	A. V. Aho et al. (Fischer-related work)	1987	*A New Parallel Algorithm for Parsing Arithmetic Infix Expressions*	Presents an algorithm for transforming arithmetic infix expressions into parse trees using precedence information.	Relevant to parsing arithmetic expressions and constructing their syntax structure.
4	Alessandro Barenghi, Stefano Crespi Reghizzi, Dino Mandrioli, Federica Panella, Matteo Pradella	2015	*Parallel Parsing Made Practical*	Investigates scalable parsing and uses arithmetic expressions with parentheses to explain grammars, derivation trees, lexical grammars, and parsing.	Supports our use of grammar, tokens, parsing, and syntax trees for arithmetic expressions.
5	Kam-Fai Chan, Dit-Yan Yeung	2001	*Error Detection, Error Correction and Performance Evaluation in On-line Mathematical Expression Recognition*	Develops parsing techniques capable of handling lexical, syntactic, and some semantic errors in mathematical expressions.	Relevant to the error detection and reporting component of our mini compiler.

Implementation

The implementation of the **Mini Compiler for Arithmetic Expressions** can be organized into a sequence of compiler phases:
The user provides an arithmetic expression as input.
Example:
(a + b) * c
The compiler receives this expression and passes it to the lexical analysis phase.
The lexical analyzer scans the input character by character and identifies meaningful tokens.
For the expression:
(a + b) * c
the tokens are:
(
a
+
b
)
*
c
The lexer identifies the different categories of tokens, including:
Operands
Numbers
Arithmetic operators
Opening parentheses
Closing parentheses
This corresponds to the first processing stage described in the project abstract

Syntax Analysis

The syntax analyzer checks whether the sequence of tokens follows the grammar of arithmetic expressions.

A simplified grammar can be represented as:

E → E + T | E - T | T

T → T * F | T / F | F

F → (E) | id | number

For example:

(a + b) * c

is syntactically valid because the operands, operators, and parentheses occur in an appropriate arrangement.

An invalid expression such as:

a + * b

should be detected as a syntax error.

The syntax analyzer therefore verifies the correct arrangement of operators and operands as described in the project abstract.

Syntax Tree / Parse Structure

After successful parsing, the expression can be represented using a syntax tree.

For:

a + b * c

the multiplication operation has higher precedence than addition.

Conceptually:

```
      +
     / \
    a   *
       / \
      b   c
```

Semantic Analysis

The semantic analysis phase performs basic validity checks on the expression.

Examples of errors that may be checked include:

- Invalid operations
- Invalid operands
- Division by zero where it can be detected
- Incorrect expression meaning

The purpose of this phase is to ensure that an expression that is syntactically valid is also acceptable according to the basic semantic rules of the mini compiler.

Intermediate Code Generation

After successful lexical, syntax, and semantic analysis, the compiler generates an intermediate representation.

For example:

Input:

a + b * c

Three-address code:

t1 = b * c

t2 = a + t1

For:

(a + b) * c

the intermediate code can be:

t1 = a + b

t2 = t1 * c

This breaks a complex arithmetic expression into simple instructions and demonstrates the intermediate code generation phase described in the project abstract

Error Handling

The compiler should identify errors and provide suitable messages to the user.

Examples:

a + * b

Output:

Syntax Error: Invalid operator placement

Another example:

(a + b

Output:

Syntax Error: Missing closing parenthesis

Error detection and reporting is an important part of the proposed mini compiler.

(DEEMED TO BE UNIVERSITY)

Overall Implementation Flow

(Mini Compiler for Arithmetic Expression)

Input

Arithmetic Expression

`(a + b) * c`

1 Lexical Analysis

- Scans the input character by character
- Identifies tokens (numbers, operators, parentheses, identifiers)

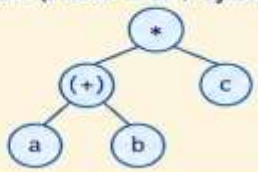
Output (Tokens)

`[ ( , a , + , b , ) , * , c ]`

2 Syntax Analysis

- Checks grammar rules
- Verifies correct arrangement of operands and operators

Output (Parse Tree / Syntax Tree)



3 Semantic Analysis

- Performs basic validity checks
- Detects invalid operations (e.g., division by zero, type errors)

Output

No semantic errors (if valid)
or appropriate error message

4 Intermediate Code Generation

- Converts the expression into three-address code (TAC)
- Breaks complex expressions into simpler instructions

Output (Three-Address Code)

```
t1 = a + b
t2 = t1 * c
```

5 Error Handling

- Identifies syntax and semantic errors
- Provides meaningful error messages

Example Error Output

Syntax Error:
Missing closing parenthesis

6 Final Output

- Intermediate code / Error message (as per the input)

Advantages

- **Easy to understand:** The mini compiler provides a simple way to understand the basic phases of compiler design.
- **Step-by-step processing:** It processes arithmetic expressions through lexical analysis, syntax analysis, semantic analysis, and intermediate code generation.
- **Error detection:** It can identify syntax and basic semantic errors and provide meaningful error messages.
- **Supports arithmetic operators:** It can process operators such as +, -, *, and /, along with parentheses.
- **Demonstrates intermediate code generation:** It converts expressions into **Three-Address Code (TAC)**, making the compilation process easier to understand.
- **Educational tool:** It provides practical knowledge of tokens, grammar, parsing, syntax trees, and compiler phases.

Limitations

- **Limited to arithmetic expressions:** The compiler is designed specifically for arithmetic expressions and does not support complete programming languages.
- **Limited operators:** It mainly handles basic arithmetic operators such as addition, subtraction, multiplication, and division.
- **Basic semantic analysis:** The semantic checking is limited to simple validity checks and invalid operations.
- **No optimization phase:** The mini compiler does not focus on advanced code optimization techniques.
- **No machine code generation:** It generates an intermediate representation such as TAC rather than executable machine code.
- **Limited error recovery:** The system focuses mainly on detecting and reporting errors rather than performing advanced error recovery.
- **Educational scope:** It is intended as a simplified model for understanding compiler concepts rather than a complete production-level compiler.

Conclusion

The Mini Compiler for Arithmetic Expressions provides a practical and simplified demonstration of fundamental compiler-design concepts. The system accepts arithmetic expressions containing operands, operators, and parentheses and processes them through different compiler phases.
The lexical analysis phase converts the input into meaningful tokens, while syntax analysis verifies the grammatical structure of the expression. Semantic analysis performs basic validity checks, and intermediate code generation converts the valid expression into a simpler representation such as Three-Address Code. The system also provides error detection and reporting for invalid expressions.
Through this project, important concepts such as tokens, lexical analysis, grammar, parsing, syntax trees, semantic analysis, intermediate code generation, and error handling can be understood practically. The project demonstrates how a high-level arithmetic expression is processed step by step and transformed into an intermediate representation. Therefore, the mini compiler serves as an educational model for understanding the basic internal working of real-world compilers.

(DEEMED TO BE UNIVERSITY)

THANKYOU